Two Monolingual Language Models from Scratch: Hindi and Nepali

Swaraj Kumar

September 2026

Abstract

[WRITE: Six to eight sentences. What you built, the single most important finding, and the number that supports it. Write this LAST, once you know what the report actually says.]

1 Introduction

[WRITE: What the project is: two independent monolingual decoder-only Transformers built from scratch, no pretrained weights or tokenizers. State your through-line here – the one claim the whole report argues. Say which language is Model H and which is Model L, and why that pairing is useful (both Devanagari, so script is not a confound).]

2 Phase 1: Corpora and Tokenizers

2.1 Collection and cleaning

[WRITE: Where the text came from and how it was cleaned. Mention the steps you can defend in a viva: NFC normalisation, zero-width and control stripping (and why ZWNJ/ZWJ are deliberately kept), pure-Latin token stripping, malformed-Unicode detection via the U+FFFD ratio, short-document filtering, three-layer language ID, SHA-1 exact dedup, MinHash near-dedup.]

2.2 Splits

[WRITE: Document-level, never chunk-level, and why: chunks of one article on both sides of the split make the test score measure memorisation. Hindi 96/2/2 seed 42; Nepali 85/7.5/7.5 seed 20260817 by deterministic doc-id hash. Explain why Nepali holds out BEFORE quality filtering, and what S09 decontamination does. Be ready to defend the differing ratios.]

2.3 Tokenizers

[WRITE: Two independent tokenizers. The vocabulary-size sweep and why you chose 8,000 for Hindi and 10,000 for Nepali. Then the mistake that mattered: Nepali's BPE has no byte fallback.]

Table 1: Tokenizer sweep. Nepali's byte-fallback column is zero at every size – that is the defect, and it is why its unknown-token rate is not.

Language	Vocab	chars/token	fertility	unk rate	byte fallback
Hindi	5,000	3.3230	1.4961	0.000000	0.018301
Hindi	**8,000**	**3.5622**	**1.3956**	**0.000000**	**0.019613**
Hindi	10,000	3.6601	1.3583	0.000000	0.020213
Nepali	5,000	3.7882	1.6988	0.002956	0.000000
Nepali	6,000	3.9410	1.6329	0.003075	0.000000
Nepali	8,000	4.1725	1.5424	0.003256	0.000000
Nepali	**10,000**	**4.3410**	**1.4825**	**0.003387**	**0.000000**

3 Phase 2: Architecture, Pretraining, Evaluation

3.1 Architecture

[WRITE: Decoder-only, 6 layers, 8 heads, d=512, d_ff=2048, context 512, pre-norm, GELU, weight tying, learned absolute positions. Say what you implemented yourself and how you verified it. The causal-mask proof – exactly 0.000e+00 leakage – belongs here, with the reason it is exactly zero rather than merely small.]

[WRITE: Justify learned absolute over sinusoidal and RoPE, and state the consequence you accepted: block_size 512 is a hard ceiling.]

3.2 Training

[WRITE: AdamW, decay on matrices only and why. Warmup then cosine. float16 with a gradient scaler because the T4 is sm_75 and has no bfloat16. Gradient accumulation and effective batch. What a checkpoint contains and why optimizer and RNG state are in it. Then the honest bit: both runs stopped short of 15,000 steps when the 11-hour budget fired.]

Table 2: Both models. Non-embedding parameter counts are identical by design, so differences are attributable to data and tokenizer.

	Model H (Hindi)	Model L (Nepali)
Vocabulary	8,000 Unigram	10,000 BPE
Total parameters	23,273,472	24,297,472
Non-embedding parameters	**18,915,328**	**18,915,328**
Steps completed	14,520	14,440
Training tokens	475,791,360	473,169,920
Evaluated checkpoint	14,500	14,000
Test cross-entropy (nats)	3.3109	3.8433
Test perplexity	**27.41**	46.68
Test bits-per-byte	0.5382	**0.4846**
Bytes per token	8.875	11.441

3.3 Perplexity and bits-per-byte disagree

[WRITE: This is probably your strongest section. Explain why per-token perplexity is not comparable across tokenizers, show the arithmetic (4.7766 bits/token / 8.88 bytes/token = 0.5382; 5.5447 / 11.44 = 0.4846), and state what you would have wrongly concluded from perplexity alone. Include the caveat that BPB is deflated for both because Devanagari costs three bytes per character.]

3.4 Generation

[WRITE: Temperature and repetition. Greedy repeats 58–60% of 4-grams; T=1.0 drops that to 0.002 while chrF++ simultaneously improves. Explain the mechanism – temperature divides logits, greedy is the argmax limit and is deterministic, so it cannot escape a loop. Say why BLEU is near the floor here and why chrF++ is the metric to trust.]

3.5 Attention

[WRITE: Entropy against the 5.2565 uniform baseline, and why the baseline is necessary for the number to mean anything. The layer story, and the fact that both models independently learned a previous-token head at layer 3. Explain why H attends further than L in tokens but not in bytes.]

4 Phase 3: Reasoning Finetuning

4.1 Synthetic dataset

[WRITE: Four task families, how examples are generated, and how you prevent leakage: 40/10/10 disjoint entity names and 14/4/4 disjoint templates, asserted programmatically. Mention the defect you found and fixed during construction.]

4.2 Results

[WRITE: The headline: exact match goes 0 to 35.15% and 0 to 29.45%, but forced choice sits at or below the 41.67% chance rate. Say plainly what that means – the models learned an output format, not inference.]

4.3 Error analysis

[WRITE: The two-hop result. On transitive3 Model H answers the middle entity 72.6% of the time and the first entity 0.0%, while gold is 47.4/0.0/52.6. B is the only name mentioned twice. Argue what that shows. Also the resource-tier difference: off-prompt answers 0–7.2% for H against 19.2–28.8% for L.]

Table 3: Reasoning, before and after finetuning. Chance is 41.67%.

	H pretrained	H finetuned	L pretrained	L finetuned
Exact match	0.0000	**0.3515**	0.0000	**0.2945**
Forced choice	0.3980	0.3635	0.3880	0.3930
Answer-token loss	5.2439	0.9408	5.5367	1.4351
Answer-token perplexity	189.40	2.56	253.84	4.20

5 Bonus: Removing Positional Embeddings

[WRITE: One variable changed. What you expected from permutation invariance, what actually happened, and your explanation. Note the attention evidence and the previous-token head moving from layer 3 to layer 4. Be careful to present the mask-leakage account as an explanation consistent with the evidence, not as something this experiment proves.]

Table 4: Ablation, matched at step 14,500.

	Standard	No positions
Total parameters	23,273,472	23,011,328
Non-embedding	18,915,328	18,915,328
Val perplexity @ 14,500	28.22	30.11
Test perplexity	27.41	29.01
Test bits-per-byte	0.5382	0.5474
Attention mean entropy	3.2047	3.6005
Attention mean distance	62.36	77.90

6 Reflection and Limitations

[WRITE: This section is graded on honesty, and it is the part an examiner will probe hardest. Write it yourself and do not soften it. Cover at least: the runs that stopped short of budget; the Nepali tokenizer decision you would reverse; forced choice being below chance; single seed throughout; and what you would do next and why.]

Artifacts

Code: `https://github.com/CL3-410/individual-project-swarajkumar001`
Checkpoints (pretrained and finetuned, H and L): **[WRITE: paste the Google Drive link, public viewing enabled]**
Branches: `phase-1`, `phase-2`, `phase-3` (submission), `bonus-no-pos` (ablation).